

The Economics of Frontier-AI Competition: Leader Profitability under Open-Weight Catch-Up*

Pavel Kocourek

July 2026 — preliminary draft

1 The question

Frontier AI labs spend tens of billions of dollars a year training models whose capabilities an open-weight fringe reproduces within months and then sells at marginal compute cost. It is thus an open question whether the frontier labs can ever turn a profit: whatever they build is soon offered by others at a price that leaves no margin. This note sets out a small dynamic model of that race to answer it.

The model treats the frontier labs as a single leader and the open-weight ecosystem as a competitive fringe that prices at cost. The leader can then charge only for the capability increment it holds over the fringe: its earnings are proportional to the value of the lead — the gap between what its model is worth and what the fringe’s is. Under what conditions does the leader’s flow profit turn positive within roughly five years — and stay positive? The answer turns on four rates and how they combine: how fast the frontier moves, how fast the follower closes the gap, how steeply value grows with capability, and how fast the model-building bill grows.

The note mirrors the companion widget exactly: Section 2 lays out the skeleton, and Sections 3–5 follow the widget’s three levels — Basics, Dynamics, Catch-up channels — switching mechanisms on cumulatively, so nothing is ever replaced, only added.

2 Setup

Two players. The **leader** L is the set of frontier labs collapsed into one representative player training the single frontier model. The **follower** F is the open-weight / competitive fringe: it sells inference at marginal compute cost, so it earns nothing itself but disciplines the leader’s price — customers can obtain the follower’s capability essentially for free, and the leader can charge only for the increment above it.

*Preliminary and incomplete — an early draft, circulated for feedback. Written under a fellowship of the Center for AI Safety (CAIS). The note is the model companion to the interactive explorer at https://pkocourek.com/frontier_labs_profit, which implements exactly the model presented here, level by level, with the full calibration and a Monte Carlo forecasting mode. This note carries the symbolic model; calibration numbers, their sources, and sensitivity exploration live in the widget.

States. Capability is measured in orders of magnitude (OOMs) of *effective compute*: algorithmic level times physical training compute. With $a = \log_{10}(\text{algorithmic level})$ and $c = \log_{10}(\text{training compute})$,

$$x_t^i = a_t^i + c_t^i \quad (i \in \{L, F\}), \quad \Delta_t = x_t^L - x_t^F, \quad (1)$$

normalized so that the mid-2026 frontier sits at $x_0^L = 0$ — component-wise, $a_0^L = c_0^L = 0$; the follower starts a gap Δ_0 behind.

Payoff skeleton. Let $W(x)$ denote the *value index* of capability x : a dimensionless map with $W(0) = 1$, so W reads as value as a multiple of today’s frontier value. The model carries no dollar level at all: earnings and the model-building bill are each normalised by their own level *today*, so every financial quantity is measured in multiples of today’s model-building bill, and the leader’s flow profit is simply

$$\Pi_t^L = E_t - B_t, \quad (2)$$

with earnings E_t riding the value gap $W(x_t^L) - W(x_t^F)$ and the bill starting at $B_0 = 1$. The model’s whole content is in the dynamics that generate x_t^L and x_t^F , the shape of W , the cost path, and the one financial parameter the normalisation leaves behind.

3 Basics — the race, costs, and earnings

The base model holds every rate constant. The leader’s capability grows at a constant speed: compute scales at rate g_c (OOM/yr) and algorithms improve at rate g_a , so $\dot{x}^L = g_c + g_a$. The follower has no engine of its own — it moves purely by catching up, closing the gap at rate δ (per year):

$$\dot{x}_t^F = \delta (x_t^L - x_t^F) = \delta \Delta_t, \quad \text{so} \quad \dot{\Delta}_t = \dot{x}^L - \delta \Delta_t. \quad (3)$$

The observed lag of the open-weight fringe behind the frontier pins δ : with $\delta\Delta_0$ equal to the leader’s speed, the lag is constant and the gap holds at Δ_0 .

Value is exponential in capability: each additional OOM multiplies the index by 10^ν ,

$$W(x) = 10^{\nu x}, \quad (4)$$

with the slope ν measured in OOMs of value per OOM of capability.

The bill. On the cost side, the leader rents compute and pays the bill B_t of the model it is currently running, in units of today’s bill:

$$B_t = 10^{c_t^L - g_p t}, \quad (5)$$

where g_p is the hardware price-performance decline (OOM/yr) — the bill tracks the compute path at falling prices, and algorithmic progress is already counted inside x , not here. Since $c_0^L = 0$ by the capability normalisation, $B_0 = 1$: the bill is measured in multiples of itself today.

Earnings. Earnings are the rent on the lead: they ride the value gap over the at-cost follower, with the level pinned against today’s bill,

$$E_t \sim W(x_t^L) - W(x_t^F), \quad E_0 = \rho B_0, \quad (6)$$

where “earnings” means earnings *before* model-building costs. The pair is exact as it stands: the $\sim$ hides one fixed constant — the same gap valued today — and the second equation fixes it, so that equivalently $E_t = \rho B_0 [W(x_t^L) - W(x_t^F)] / [W(x_0^L) - W(x_0^F)]$. With $B_0 = 1$ this makes $E_0 = \rho$, the model’s one financial parameter, discussed next.

The coverage ratio. The headline object is the ratio of earnings to the bill,

$$\rho_t = \frac{E_t}{B_t}, \quad \Pi_t^L > 0 \iff \rho_t > 1, \quad \rho_0 = \rho. \quad (7)$$

The single financial parameter ρ is *coverage today*: operating earnings per unit of model-building outlay. Reported dollar figures are calibration evidence for it, not parameters of the model: with gross revenue R_0 , operating margin before model-building costs m , and model-building outlay $k R_0$, coverage today is $\rho = m/k \approx 0.33$ — the industry earns roughly 33 cents per dollar of model-building spend. Dollar levels never enter: scaling every reported figure leaves the model unchanged, so only the ratio m/k was ever identified, and the model now carries just it. Break-even is $\rho_t = 1$; coverage starts below one, and the question is whether it climbs.

Will the leader ever be profitable? With the gap constant and value exponential, one inequality decides it. Earnings inherit W ’s growth rate along the leader’s path, growing by the factor $10^{\nu(g_c + g_a)}$ per year, while the bill grows by $10^{g_c - g_p}$ per year; coverage rises, and break-even eventually arrives, if and only if

$$\nu(g_c + g_a) > g_c - g_p. \quad (8)$$

At the widget’s calibration — compute scaling near $3.2\times$ per year, the bill growing near $2.35\times$ per year, value about $2.1\times$ per OOM — earnings grow about $2.2\times$ per year and (8) *fails*: coverage falls, and the base model’s verdict is that break-even never comes. Because every rate is constant, that is one verdict for all time. It rests, however, on compute scaling at $3.2\times$ per year forever — exactly the assumption the Dynamics level drops. Dropping it does not settle the question either way: a compute slowdown bends the bill, but it also slows the frontier and lets the follower close the gap — and earnings live on the gap — while accelerating algorithmic progress pulls in the opposite direction. Whether break-even ever arrives becomes a quantitative race between these forces.

4 Dynamics

The Dynamics level turns on two opposing forces — compute growth slows down while algorithmic progress speeds up — together with a build lag that starts to matter once compute growth bends, and a bend in the value curve itself. The follower, earnings, and profit carry over from Basics unchanged.

The transition curve Γ

Three of the model’s mechanisms are *transitions*: something moves from one steady value to another, over time or over capability. Two appear in this section, and a third with the follower’s compute in the next; all use the same primitive, stated once: the logistic S-curve, dialled in four *observables*,

$$\Gamma(u; y(0), y_\infty, u_{mid}, q_0) : \quad (9)$$

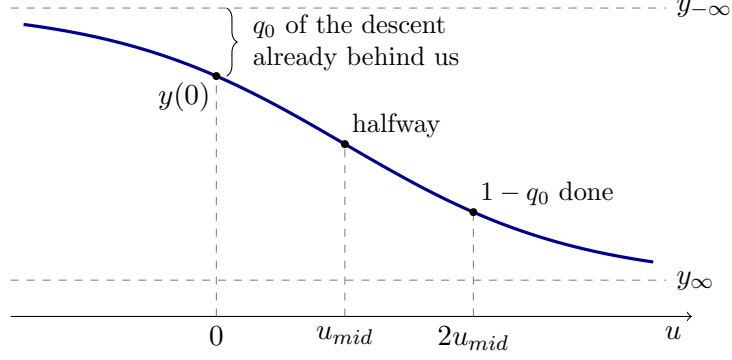


Figure 1: The transition curve $\Gamma(u; y(0), y_\infty, u_{mid}, q_0)$, drawn for a falling transition (the compute slowdown’s shape) with $q_0 = 0.25$. The four arguments are observables; the pre-transition plateau $y_{-\infty}$ and the steepness are derived (Appendix A), so the curve passes through today’s value $y(0)$ at $u = 0$ for any choice of u_{mid} and q_0 .

today’s value $y(0)$, the destination y_∞ , the midtime u_{mid} at which the transition is halfway done, and the position q_0 — the fraction of the transition already completed today (necessarily $q_0 < \frac{1}{2}$: the midpoint is still ahead). Figure 1 shows the shape. The plateau the curve came from and its steepness are *not* free parameters: they are derived so that the curve passes through today’s value, already q_0 of the way along, at $u = 0$ (Appendix A has the closed form). So $\Gamma(0) = y(0)$ identically — an observation never depends on a scenario parameter, and moving u_{mid} or q_0 says where the transition *started*, never what it is now. By the logistic’s symmetry the curve is $1 - q_0$ done at $2u_{mid}$, so a transition occupies roughly the window $[0, 2u_{mid}]$; and whenever $y_\infty = y(0)$ it is exactly constant.

Force 1 — the compute slowdown

Today’s compute scaling cannot persist: power, fabrication capacity, and capital all bind. Compute growth now rides the transition curve down to a long-run floor $g_{c\infty}$, half-done at t_{mid} :

$$\dot{c}_t^L = g_{c,t} = \Gamma(t; g_c, g_{c\infty}, t_{mid}, q_0^c). \quad (10)$$

The anchor is *today’s* realised growth g_c — the same g_c as in Basics, held fixed by construction — so positing that the slowdown is further along (q_0^c larger) raises the pre-slowdown plateau it came down from, never today’s growth. Whenever $g_{c\infty} = g_c$ the curve is exactly constant and the base model never sees the shape.

Force 2 — algorithmic progress speeds up

The base model’s constant algorithmic rate g_a becomes a *produced* quantity: a CES aggregate of researcher effort, accelerated by AI assistance, and experiment throughput, which moves with compute growth:

$$\dot{a}_t^L = g_a \left[(1 - \alpha) \left(\frac{\psi(x_t^L)}{\psi(0)} \right)^\eta + \alpha \left(\frac{g_{c,t}}{g_c} \right)^\eta \right]^{1/\eta}, \quad \psi(x) = 1 + \beta_0 10^{\gamma x}. \quad (11)$$

Both inputs are normalized to one at $t = 0$, so $\dot{a}_0^L = g_a$ remains today's rate. The function ψ is the AI-assistance multiplier: β_0 is the speedup AI delivers to its own R&D today, and γ is how strongly that assistance compounds with capability — $\gamma = 0$ freezes AI assistance at today's level. The exponent η sets how researchers and experiment compute substitute, through the elasticity $\sigma = 1/(1 - \eta)$: $\eta = 0$, the base case, is Cobb–Douglas, the convention the recursive-self-improvement literature adopts and the case in which α equals the compute cost share under cost minimization; $\eta = 1$ makes them perfect substitutes, so the bracket is a weighted average and a compute slowdown barely bites; $\eta < 0$ makes them complements, so the scarcer input rules. α is the experiment-compute weight.

With $\gamma > 0$ the ψ -feedback makes the frontier law formally super-exponential: for large enough γ the path reaches a finite-time singularity inside the horizon, and the widget flags when the feedback's share of algorithmic progress stops being small. More consequential for the race, the α -weighted term couples algorithmic progress to compute growth, so the slowdown drags the algorithmic engine down with it rather than leaving it untouched.

Value in logs — the slope transition

Write value in logs, $w = \log_{10} W$, so $W(x) = 10^{w(x)}$ with $w(0) = 0$ (the index anchor $W(0) = 1$). What the Dynamics add is that the *slope* of w rides the transition curve — its second use, this time over capability rather than time:

$$w'(x) = \Gamma(x; \nu, \nu_\infty, x_{mid}, q_0^w). \quad (12)$$

Each OOM of capability is worth $10^\nu \times \text{today}$ ($w'(0) = \nu$, the same ν as in Basics), easing to a floor of $10^{\nu_\infty} \times$ once the transition is done, halfway at x_{mid} (OOMs above today's frontier). There is no hard ceiling: value grows without bound at the floor slope, so each further OOM is worth at least $10^{\nu_\infty} \times$.

The shape of the value curve is the model's pivotal assumption. On the steep stretch a defended, constant gap earns *growing* rents — the leader harvests its lead. Once the slope eases, the same gap's rents grow ever more slowly: commoditization in all but name, if ν_∞ is low. Where the transition sits (x_{mid}) and where the slope lands (ν_∞) are therefore decisive *as a pair*, and asymmetrically so: x_{mid} is the gate. If the midpoint sits far above the capability the frontier reaches within the horizon, the whole path runs on the steep stretch and the floor slope barely registers, however low it is set; bring the midpoint within reach and the same choice of ν_∞ swings the outcome between harvest and commoditization. So the two are explored jointly, with x_{mid} deciding whether ν_∞ matters at all.

The frontier speed race

The frontier's speed is the sum of the two forces,

$$\dot{x}_t^L = \dot{c}_t^L + \dot{a}_t^L, \quad (13)$$

and they pull in opposite directions: the compute slowdown brakes, the ψ -feedback accelerates — and through the α -weighted term the brake also drags on the accelerator. Whether the frontier speeds up or slows down is a calibration question, not a modelling one. One consequence carries forward: with δ still pinned by the observed lag, a decelerating leader means a *closing* gap. The next level gives the follower an engine of its own and makes the gap stationary again.

5 Catch-up channels

The follower now gets an engine of its own — its own algorithmic rate g_a^F and its own compute path, the same transition-curve family with its own parameters,

$$\dot{c}_t^F = \Gamma(t; g_c^F, g_{c\infty}^F, t_{mid}^F, q_0^F), \quad (14)$$

and the single catch-up rate δ unpacks into two channels. What catches up is the follower’s *algorithmic* level, because compute cannot be copied:

$$\dot{a}_t^F = g_a^F + \underbrace{\delta_{dev} (a_t^L - a_t^F)}_{\text{developed-model channel}} + \underbrace{\delta_{rel} (x_t^L - x_t^F)}_{\text{released-model channel}}. \quad (15)$$

The *developed-model* channel δ_{dev} is methods diffusion — espionage, papers, talent mobility — so it runs on the algorithmic gap and operates even if the leader releases nothing. The *released-model* channel δ_{rel} is distillation of what the leader’s deployed model can do, so it runs on the capability gap. Both gaps are assumed positive in (15) — the leader is ahead; in the implementation each gap is floored at zero, so imitation dies at parity and the follower never un-learns. Both cross terms measure imitation only; self-improvement gains run on each player’s own engine.

The initial gap splits between the two states,

$$a_0^F = -\text{split} \cdot \Delta_0, \quad c_0^F = -(1 - \text{split}) \Delta_0. \quad (16)$$

The calibration treats the channel rates and the initial split differently. The observed open-closed lag pins the $t = 0$ *transfer*: with the follower supplying most of its own speed, the two channels together need only close the wedge between the leader’s speed and the follower’s own engine, $\dot{x}_0^L - (g_a^F + \dot{c}_0^F)$, spread over the gap Δ_0 . Because the developed-model channel runs on the algorithmic gap, which at $t = 0$ is $\text{split} \cdot \Delta_0$ rather than Δ_0 , what the lag pins is the *weighted* sum $\text{split} \cdot \delta_{dev} + \delta_{rel}$, not the bare total. The ratio $\delta_{dev} : \delta_{rel}$ has no observable of its own and is held at its calibrated value, so the lag fixes the pair up to that one direction. The free, explored parameter is the initial-gap split above: because the two channels run on different gaps — the algorithmic gap and the full capability gap — moving it redistributes the pinned transfer between them and reshapes the follower’s path.

6 Outputs

The model is now complete: the leader’s path from (10)–(11), the follower’s from (14)–(15), value from (12), and profit from the skeleton (2) with cost (5).

Headline. The headline output is the path of the leader’s flow profit Π_t^L : does it turn positive within roughly five years, and does it *stay* positive? Equivalently, does the coverage ratio $\rho_t = E_t/B_t$ cross one — starting from its handicap $\rho_0 = \rho$?

The long-run benchmark. When the system settles — bounded growth regime, constant gap — earnings grow at $\nu_\infty \dot{x}_\infty^L$ and the bill at $g_{c\infty} - g_p$ (both in OOM/yr), so in the long run the leader harvests if and only if

$$\nu_\infty \dot{x}_\infty^L > g_{c\infty} - g_p. \quad (17)$$

The normalisations and the build lag are constant factors and drop out of growth rates. This limit favors harvest at strikingly low floor slope ν_∞ : once compute growth has fallen to its floor, even modest value scaling outruns the bill. The battleground is therefore the *transition* — today the bill’s growth outruns earnings growth for moderate ν , so Π_t^L dips before recovering, and the five-year sign question is about levels and the slowdown timing t_{mid} , not the limit.

What this note does not do. Every parameter above is calibrated to a stated observable or explicitly flagged as free, and the contested ones carry ranges rather than points; the companion widget¹ exposes the same three levels — Basics, Dynamics, Catch-up channels — documents every calibration target, and draws jointly from all ranges in a Monte Carlo mode to turn the headline question into a forecast distribution.

Appendix A The transition curve in closed form

Let σ be the sigmoid function, $\sigma(z) = 1/(1 + e^{-z})$, rising from 0 to 1 with $\sigma(0) = \frac{1}{2}$, $\sigma(-z) = 1 - \sigma(z)$, and inverse the log-odds $\sigma^{-1}(f) = \ln \frac{f}{1-f}$, read on a fraction f . The transition curve is the sigmoid run between two plateaus,

$$\Gamma(u; y(0), y_\infty, u_{mid}, q_0) = y_{-\infty} + (y_\infty - y_{-\infty}) \sigma(s(u - u_{mid})), \quad (18)$$

with the steepness s and the pre-transition plateau $y_{-\infty}$ derived from the four dialled observables by two identities.

The steepness, from the position. The fraction of the transition completed at $u = 0$ is $\sigma(-s u_{mid}) = q_0$, so

$$s u_{mid} = \ln \frac{1 - q_0}{q_0}, \quad (19)$$

the log-odds of the journey remaining. The curve then reads q_0 done at $u = 0$, one half at u_{mid} , and — by the sigmoid’s symmetry — $1 - q_0$ at $2u_{mid}$, for any q_0 .

The plateau, from today’s value. Today’s value is the position-weighted average of the two plateaus, $y(0) = (1 - q_0) y_{-\infty} + q_0 y_\infty$; un-mixing it gives

$$y_{-\infty} = \frac{y(0) - q_0 y_\infty}{1 - q_0}. \quad (20)$$

As $q_0 \rightarrow 0$ the plateau is today’s value — nothing has happened yet. For a slowdown ($y_\infty < y(0)$) it returns $y_{-\infty} > y(0)$, correctly: we have already come part of the way down.

Substituting both identities into (18) confirms $\Gamma(0) = y(0)$ for every u_{mid} and q_0 .

¹https://pkocourek.com/frontier_labs_profit